

Week 11 — Final Project Part II

Code the Dream · Python Intro

Session Overview

Explore — ~25 min

- Two extension tracks: what each produces
- Choosing between Option A and Option B
- Option A: data visualization with matplotlib
- Option B: data cleaning and CSV export
- Rubric requirements for each track

Apply — ~35 min

- Code-along: Option A — bar chart with matplotlib
- Code-along: Option B — cleaned CSV with DictWriter
- Course wrap-up

Two Extension Tracks

This week you add one meaningful layer on top of your Week 10 project.

	Option A	Option B
What you build	A chart that answers a question about your data	A cleaned, exported CSV from your API data
New tool	<code>matplotlib</code>	<code>csv.DictWriter</code>
Deliverable	A PNG image file	A sample CSV file
README section	<code>## Visualization</code>	<code>## Data Cleaning Decisions</code>

Choosing Your Track

Choose Option A if:

- Your data has numeric values worth comparing visually
- You want to communicate a trend or comparison
- Example: top 10 countries by population as a bar chart

Choose Option B if:

- Your data has missing fields, inconsistent types, or records to filter
- You want to produce a clean, exportable dataset
- Example: countries with missing capitals removed, population normalized

Option A: matplotlib Basics

Install and import matplotlib, then build a chart.

```
import matplotlib.pyplot as plt

labels = ["Finland", "Norway", "Sweden"]
values = [5530719, 5421242, 10379295]

plt.bar(labels, values)
plt.title("Population: Nordic Countries")
plt.ylabel("Population")
plt.tight_layout()
plt.savefig("chart.png")
```

Check for Understanding #1

Your chart rubric says it must "answer a specific question you pose about your data."

Which is a better chart title?

- A) "Countries Data"
- B) "Top 10 Most Populous Countries in Asia"
- C) "Population Chart v2"
- D) "My Final Project"

Option A: Connecting to Your Data

Pull values from your Week 10 list of dicts.

```
def make_population_chart(countries):  
    top10 = sorted(  
        countries,  
        key=lambda c: c["population"],  
        reverse=True  
    )[:10]  
  
    names = [c["name"] for c in top10]  
    pops  = [c["population"] for c in top10]  
  
    plt.barh(names, pops)  
    plt.title("Top 10 Most Populous Countries")  
    plt.xlabel("Population")  
    plt.tight_layout()  
    plt.savefig("top10_population.png")
```

Option B: Data Cleaning Decisions

Before exporting, decide what to do with each field.

Field problem	Approach
Missing value	Fill with <code>"N/A"</code> or <code>0</code> , or exclude the row
Wrong type (string instead of number)	Convert with <code>int()</code> or <code>float()</code>
Invalid record (population = 0)	Filter it out before export

Option B: csv.DictWriter

Export a list of dicts to a CSV file.

```
import csv

def export_to_csv(countries, filename="clean_countries.csv"):
    fields = ["name", "capital", "region", "population"]
    with open(filename, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fields)
        writer.writeheader()
        writer.writerows(countries)
    print(f"Exported {len(countries)} rows to {filename}")
```

Check for Understanding #2

When exporting with `csv.DictWriter`, what does `writeheader()` do?

- A) Writes all the data rows
- B) Writes the first row of column names
- C) Opens the file for writing
- D) Validates that the dicts match the fieldnames

Check for Understanding #3

What should the `## Data Cleaning Decisions` section in your README describe?

- A) The Python version you used
- B) Which fields you included, what you did with missing or invalid values, and any type conversions
- C) How to run the program
- D) The API you chose and its documentation URL

Time to Apply

Let's build each extension track.

Mentor 2 takes over

Assignment 11 Overview

Both tracks require:

- Extension logic in reusable functions
- At least one standard library module beyond `csv` (`os`, `datetime`)
- Incremental commits across both weeks
- A 2–4 min video demo in your PR description and README

Option A adds:

- A chart answering a stated question + labels + saved as PNG
- `## Visualization` section in README

Option B adds:

- Cleaned CSV exported with `csv.DictWriter`

`## Data Cleaning` section in README

Code-Along: Option A — Bar Chart

Build a chart function that takes your list of dicts.

```
import matplotlib.pyplot as plt

def plot_top_countries(countries, n=10):
    top = sorted(countries,
                 key=lambda c: c["population"],
                 reverse=True)[:n]
    names = [c["name"] for c in top]
    pops  = [c["population"] / 1_000_000 for c in top]

    plt.figure(figsize=(10, 6))
    plt.barh(names[::-1], pops[::-1])
    plt.xlabel("Population (millions)")
    plt.title(f"Top {n} Most Populous Countries")
    plt.tight_layout()
    plt.savefig("top_countries.png")
    print("Chart saved to top_countries.png")
```

Code-Along: Option B — Clean + Export

Filter, clean, and write to CSV with a timestamp.

```
import csv, os
from datetime import datetime

def export_clean_data(countries, filename=None):
    if filename is None:
        ts = datetime.now().strftime("%Y%m%d")
        filename = f"clean_countries_{ts}.csv"

    valid = [c for c in countries if c["population"] > 0]
    fields = ["name", "capital", "region", "population"]

    with open(filename, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fields)
        writer.writeheader()
        writer.writerows(valid)
    print(f"Exported {len(valid)} rows to {filename}")
```

Extension: Connect to main()

Add your extension track as a CLI option.

```
# In main(), add a new menu option:  
elif choice == "4":  
    plot_top_countries(data)      # Option A  
    # OR  
    export_clean_data(data)      # Option B
```

This is how the extension integrates cleanly with your Week 10 CLI without changing any existing code.

Congratulations — You Made It!

Over 11 weeks you've built:

- Command-line scripts, interactive programs, and a full CLI app
- Proficiency with Python's core data structures and control flow
- File I/O, CSV parsing, error handling, and virtual environments
- A live API-powered application — from scratch

These skills are the foundation for data science, web development, automation, and more.

Wrap-Up & What's Next

To finish strong:

- Complete your chosen extension track this week
- Update `requirements.txt` if you added new packages
- Record your 2–4 min video demo
- Write the required README section (Visualization or Data Cleaning Decisions)
- Submit via pull request

Coming up in Code the Dream:

- Web development (HTML, CSS, JavaScript)
- Frameworks: Django or React
- The skills you built here transfer directly